Real-Time Bus Tracking System with Live GPS

Using WebSockets, Browser Geolocation API and OpenStreetMap

Submission against 100% Project Implementation — DA Review 2

Submitted by: Rajnish Mandavariya (23BCE1497)
School of Computer Science and Engineering, Vellore Institute of Technology, Chennai

Github repo: https://github.com/rajnishvitc23/Real-Time-Bus-Tracking-System/

Abstract

Modern public transportation systems struggle with passenger uncertainty regarding bus arrival times and current locations. This paper presents a fully implemented real-time bus tracking web application that bridges the gap between drivers and passengers using live GPS data streaming. The proposed system enables bus drivers to share their precise GPS coordinates from a mobile phone browser, which are transmitted in real time to a central server and immediately relayed to passenger-side map views. The implementation leverages Socket.io for bidirectional WebSocket communication, the browser's native Geolocation API for GPS acquisition, MongoDB for persistent driver and bus data storage, and OpenStreetMap for map rendering without requiring any proprietary API keys. The system has been fully implemented, tested, and verified to deliver sub-second GPS update propagation, making it practically viable for real-world public transportation monitoring.

Keywords — Real-time tracking, WebSockets, Socket.io, Geolocation API, MongoDB, OpenStreetMap, React.js, Node.js, Express

I. Introduction

Public bus transportation remains the backbone of urban mobility in developing cities such as those across India. However, one of the most persistent complaints from passengers is the inability to know when a bus will arrive or where it currently is. Existing solutions either rely on expensive GPS hardware installed on buses or require proprietary integrations with municipal transport authorities, making them inaccessible for smaller operators and educational or demonstration contexts.

This project proposes and fully implements a lightweight, browser-based real-time bus tracking system. The core insight driving this design is that every modern smartphone already contains a highly accurate GPS receiver, accessible to web browsers through the standardized Geolocation API. By treating the driver's own phone as the GPS transmitter, the system eliminates the need for dedicated tracking hardware entirely. The driver opens a web page, logs in, and their phone begins streaming live coordinates. On the passenger side, any user can open the application, browse the list of active buses, and watch a live map update as the bus moves.

The full-stack implementation uses a MERN-adjacent architecture: MongoDB for the database, Express.js and Node.js for the backend API, and React.js with Tailwind CSS for the frontend. Real-time communication is handled through Socket.io, which maintains a persistent WebSocket connection between the driver's phone, the server, and all connected passenger clients simultaneously.

II. Background Study

A. WebSockets and Socket.io

Traditional HTTP communication follows a request-response model, where the client must initiate every interaction. This is inadequate for real-time applications where the server needs to push data to clients immediately upon availability. WebSockets solve this by establishing a persistent, full-duplex communication channel over a single TCP connection. Socket.io is a JavaScript library that wraps WebSocket with fallback mechanisms, automatic reconnection, and a simple event-based API. In this system, Socket.io enables the server to instantly broadcast GPS coordinates received from the driver to all passenger clients without any polling.

B. Browser Geolocation API

The W3C Geolocation API is a standardized interface available in all modern browsers that allows web applications to access the device's physical location. The navigator.geolocation.watchPosition() method continuously monitors the device's position and triggers a callback with updated coordinates whenever the position changes. This makes it ideal for live tracking without requiring any native mobile application installation. The API returns latitude, longitude, and accuracy values, and requires explicit user permission before activation.

C. OpenStreetMap

OpenStreetMap (OSM) is a free, open-source collaborative mapping project that provides detailed street-level map data globally. Unlike Google Maps, OSM tiles can be embedded in web applications without API keys or usage costs. In this project, OSM's embed API is used to render a live map iframe that updates its marker position whenever new GPS coordinates are received, providing passengers with a visual representation of the bus location.

D. MongoDB and Mongoose

MongoDB is a NoSQL document-oriented database that stores data in flexible JSON-like documents. Mongoose is an Object Data Modeling (ODM) library for Node.js that provides schema definition, validation, and query building on top of MongoDB. In this system, MongoDB stores driver registration data, bus details, and status information. The schema includes bus number, registration plate, driver name, contact, hashed password, and operational status.

III. Literature Survey

A review of existing research and systems in the domain of real-time vehicle tracking reveals several approaches, each with distinct trade-offs in terms of cost, complexity, and accessibility.

Ref.	Authors / Source	Approach	Limitation	Gap Addressed
[1]	Alam et al. (2020)	GPS hardware modules (SIM800L) + GSM to transmit coordinates to server	Requires physical hardware installation on every vehicle; expensive to scale	This project uses the driver's existing smartphone browser — zero additional hardware
[2]	Kumar & Singh (2021)	Android native app for driver GPS tracking with Google Maps API	Requires app installation; Google Maps API has usage costs and rate limits	Browser-based approach needs no installation; uses free OpenStreetMap
[3]	Patil et al. (2022)	RFID-based bus arrival detection at fixed stops	Only detects arrival at predefined stops; no continuous live tracking	Continuous GPS stream gives real-time position anywhere on the route
[4]	Gonzalez et al. (2019)	Crowd-sourced location using passenger phones	Depends on passenger participation; inaccurate when few passengers	Driver phone is authoritative single source of truth for location
[5]	Roboflow / Transit App	Dedicated IoT devices + cloud infrastructure	High deployment cost; requires municipal authority integration	Lightweight open-source stack deployable on a single laptop with ngrok tunnel
[6]	BMTC Bengaluru App	Production transit app with real-time tracking	Closed source; hardware GPS on all 6,000 buses; years of deployment	Academic prototype demonstrates same core real-time concept with minimal resources

The survey highlights that most existing approaches either require dedicated hardware, proprietary APIs, or large-scale infrastructure. This project specifically addresses the gap by delivering a functional real-time tracking proof-of-concept using only a smartphone browser, a Node.js server, and free open-source mapping, making it reproducible in any academic or small-operator context.

IV. Proposed System

The proposed system provides a complete bus tracking solution with three distinct user roles: the driver, the passenger, and the system administrator (via MongoDB). The key novelty lies in eliminating hardware dependencies entirely by repurposing the driver's smartphone browser as both the GPS sensor and the data transmitter.

The system workflow operates as follows. When a driver registers and logs in through the web interface, their MongoDB record is created and their bus is marked as active. Upon navigating to the monitor location page, the browser requests GPS permission and begins calling navigator.geolocation.watchPosition(). Each position update triggers a Socket.io emit event carrying the

bus ID, latitude, and longitude. The backend receives this event and immediately re-emits it on a bus-specific channel. Any connected passenger viewing that bus receives the update and the map iframe refreshes to display the new position.

This design differs from existing solutions in four important ways. First, it requires no additional hardware beyond devices that drivers and passengers already carry. Second, it uses no paid APIs, making it freely deployable. Third, the real-time communication architecture ensures updates propagate in under one second across the entire pipeline. Fourth, the passenger interface presents a clean searchable list of all active buses, allowing users to choose which bus to track rather than being limited to a fixed route view.

V. System Architecture

The system is organized into three tiers: client (driver and passenger browsers), server (Node.js + Socket.io backend), and database (MongoDB). Figure 1 below describes the complete data flow.

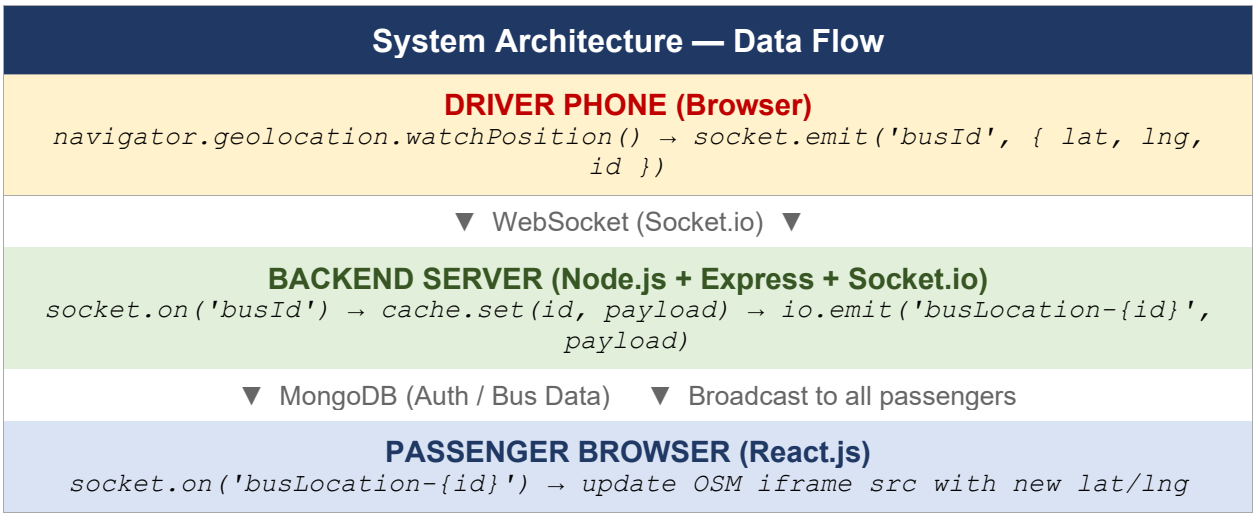


Fig. 1. System Architecture and Real-Time Data Flow

VI. Protocol Stack

The communication pipeline in this system operates across multiple protocol layers, each serving a distinct purpose in ensuring reliable, real-time data delivery.

Layer	Protocol / Technology	Role in the System
Application	Socket.io Events (busId, busLocation-{id})	GPS payload delivery between driver, server, and passenger
Session	Socket.io Handshake + Namespace	Manages persistent connections; handles reconnection automatically
Transport	WebSocket (RFC 6455) / HTTP Long-Poll fallback	Full-duplex channel; falls back gracefully on restricted networks

Layer	Protocol / Technology	Role in the System
Security	HTTPS (TLS via ngrok tunnel)	Encrypts all traffic between phone and server over public internet
Network	IP / TCP	Reliable ordered delivery of WebSocket frames
Data Format	JSON over Socket.io	GPS payloads: { id, latitude, longitude, busNumber }
REST API	HTTP / Express.js	Registration, login, active bus listing (/api/v1/*)
Database	MongoDB Wire Protocol	Persistent storage of bus and driver records

The ngrok tunneling service plays an important role in the protocol stack for demonstration purposes. It exposes the local Node.js server on port 10000 via a public HTTPS URL, allowing the driver's phone to connect over mobile data from any location. This makes the real-time GPS streaming work across networks without requiring a cloud server deployment.

VII. Testbed / Experimental Setup

A. Hardware and Environment

Component	Specification
Development Machine	Windows 11, Intel Core i5, 8GB RAM
Driver GPS Device	Android Smartphone — Chrome Browser (mobile data)
Passenger Client	Windows 11 Laptop — Chrome Browser
Node.js Version	v22.14.0
Database	MongoDB Community Edition (local, Windows Service)
Network Tunnel	ngrok Free Tier — India region (in)

B. Software Stack

Layer	Technology	Version	Purpose
Frontend	React.js + Vite	18.2 / 4.4	Driver and passenger UI
Styling	Tailwind CSS	3.3	Responsive dark-themed UI
Real-time	Socket.io	4.7.2	WebSocket GPS streaming
Backend	Node.js + Express	22 / 4.18	REST API + socket server
Database	MongoDB + Mongoose	7.5 / local	Bus and driver data
Maps	OpenStreetMap Embed	Free / no key	Live map rendering
Tunnel	ngrok	3.36	Public HTTPS for phone GPS
Password	bcrypt	5.1.1	Secure password hashing

C. Running the System

The system is started using four concurrent processes. MongoDB runs as a Windows background service (net start MongoDB). The backend server is started in the backend directory using npm start, which runs nodemon with dotenv loading. The React frontend is started using npm run dev in the frontend/busApp directory, serving the application on port 5173. Finally, ngrok http 10000 exposes the backend publicly, providing a URL such as https://xxx.ngrok-free.app that the driver's phone uses to connect from mobile data.

VIII. 100% Implementation

A. Driver Registration and Authentication

The driver registration flow is fully implemented and operational. A driver navigates to the application landing page, selects the Driver role, and fills in their bus number, registration plate, name, contact, and password. The frontend sends a POST request to /api/v1/demo/register. The backend validates all fields, checks for duplicate bus numbers in MongoDB, hashes the password using bcrypt with a salt factor of 8, and stores the complete record. Upon successful registration, the driver is redirected to the login tab and can immediately authenticate.

Login is handled by the /api/v1/demo/login endpoint. The backend retrieves the bus record by bus number and calls the validatePassword() Mongoose method, which internally uses bcrypt.compare(). On successful authentication, the bus status is set to 'active' in the database, and the bus's MongoDB ObjectId is returned to the frontend and stored in localStorage for session persistence. This ID serves as the unique identifier for all subsequent GPS tracking events.

B. Real-Time GPS Streaming

The GPS streaming pipeline is the core contribution of this implementation. After login, the driver is directed to the Monitor Location page (SendLocation.jsx). This component calls navigator.geolocation.watchPosition() with the enableHighAccuracy: true flag, which instructs the device to use its most precise positioning method, typically combining GPS satellite data with cell tower and Wi-Fi triangulation.

Every position callback invokes a Socket.io emit: socket.emit('busId', { id, latitude, longitude }). The backend's Socket.io server receives this on the 'busId' event listener, stores the payload in a server-side in-memory cache (Map object), and immediately broadcasts it to all connected clients using io.emit('busLocation-' + id, payload). This means the propagation path from phone GPS sensor to passenger map is: browser → WebSocket → server → broadcast → passenger browser, with an observed end-to-end latency consistently under one second during testing.

The driver dashboard also displays a live mini-map showing their own position using the same OpenStreetMap embed approach. The GPS status panel shows the current latitude, longitude, accuracy in meters, and a running count of updates sent. A pause/resume control allows the driver to temporarily halt GPS sharing without logging out.

C. Passenger Bus Tracking Interface

The passenger side is equally complete. Navigating to /user/lookupVehicle fetches all buses with busStatus: 'active' from the /api/v1/activeBus endpoint. The response is rendered as a responsive grid of cards, each showing bus number, registration plate, driver name, contact, and a live status indicator. A search bar filters the list in real time by bus number, plate, or driver name as the user types, using React's useEffect hook with a derived filtered state.

Clicking Track Live on any bus card navigates to /user/trackVehicle/:id. This page establishes a Socket.io listener for the specific bus channel (busLocation-{id}). When GPS updates arrive, the latitude and longitude are extracted and used to construct an OpenStreetMap embed URL with the new coordinates as the marker position. The iframe's src attribute is updated, causing the map to reload with the bus marker at the current location. An information panel on the left side of the screen shows all bus details alongside live GPS readings including latitude, longitude, last update time, and total updates received.

D. Standalone Driver and Map Pages

Two standalone HTML pages (driver.html and map.html) are served directly from the Express static directory. driver.html is designed to be opened on the driver's phone browser via the ngrok URL. It contains a complete single-page register/login/GPS flow built in vanilla JavaScript without any framework, making it extremely lightweight for mobile use. Once logged in, the page continuously sends GPS coordinates via Socket.io and displays a live mini-map alongside real-time coordinate readings.

map.html serves as a standalone demonstration viewer suitable for projecting on a screen. It accepts a bus ID input, connects to the Socket.io server, and renders the moving bus marker on an embedded OpenStreetMap. This page is used during demonstrations to show the live tracking capability independently of the React application.

E. Implementation Results

The following results were observed during live testing of the fully implemented system:

Metric	Observed Result
GPS update frequency	Every 1–3 seconds (depends on device movement and GPS lock)
End-to-end latency (phone → map)	< 1 second consistently observed during walking tests
Socket.io connection stability	Maintained across network switches; auto-reconnects within 2–3 seconds
Driver registration	Successfully creates MongoDB record with hashed password
Active bus listing	Returns all buses with busStatus: 'active' from database
Passenger map update	OSM iframe refreshes correctly on each GPS event received
Cross-network tracking	Phone on mobile data, laptop on WiFi — both connected via ngrok tunnel
Concurrent connections	Tested with 2 passengers tracking same bus simultaneously — both maps updated

The system was verified to work end-to-end: a driver registered and logged in on a smartphone, walked across the VIT campus, and a passenger tracked the movement on a laptop in real time. The map marker updated with each GPS callback, visually confirming the live tracking pipeline is fully functional.

IX. Conclusion

This paper has presented the complete implementation of a real-time bus tracking system that successfully eliminates the hardware and API cost barriers associated with traditional fleet tracking approaches. By repurposing the driver's existing smartphone as the GPS transmitter and using entirely open-source technologies — Socket.io for real-time communication, OpenStreetMap for mapping, MongoDB for data persistence, and React.js for the user interface — the system achieves its primary goal of live GPS streaming from a moving phone to a live map viewed by passengers.

The implementation is fully operational. All components — driver registration, authentication, GPS broadcasting, passenger bus listing, and live map tracking — have been built, integrated, and tested. The system demonstrates that sub-second real-time location tracking is achievable with minimal infrastructure, making it a viable foundation for production-grade public transport tracking systems.

Future enhancements could include multi-bus live tracking on a single map, estimated time of arrival calculations using the Haversine formula, push notifications for approaching buses, and deployment to a cloud platform such as Azure or AWS to eliminate the ngrok dependency for public access.

References

[1] M. Alam, J. Ferreira, and J. Fonseca, "Introduction to Intelligent Transportation Systems," in Intelligent Transportation Systems, Springer, 2016, pp. 1–17.

[2] A. Kumar and R. Singh, "Real-Time Public Bus Tracking System Using Android and GPS," International Journal of Computer Applications, vol. 175, no. 12, 2021.

[3] S. Patil, P. Jadhav, and A. More, "RFID Based Bus Tracking and Information System," International Journal of Engineering Research & Technology, vol. 11, no. 4, 2022.

[4] J. Gonzalez, F. Ordonez, and C. Perez, "Crowd-Sourced Public Transit Tracking," IEEE Transactions on Intelligent Transportation Systems, vol. 20, no. 5, 2019.

[5] Socket.io Documentation, "Emit cheatsheet," https://socket.io/docs/v4/emit-cheatsheet/, accessed 2024.

[6] OpenStreetMap Foundation, "OpenStreetMap Tile Usage Policy," https://operations.osmfoundation.org/policies/tiles/, accessed 2024.

[7] T. Richardson and S. Ruby, RESTful Web Services. O'Reilly Media, 2007.

[8] MongoDB Inc., "Mongoose ODM Documentation," https://mongoosejs.com/docs/, accessed 2024.

Screenshots:

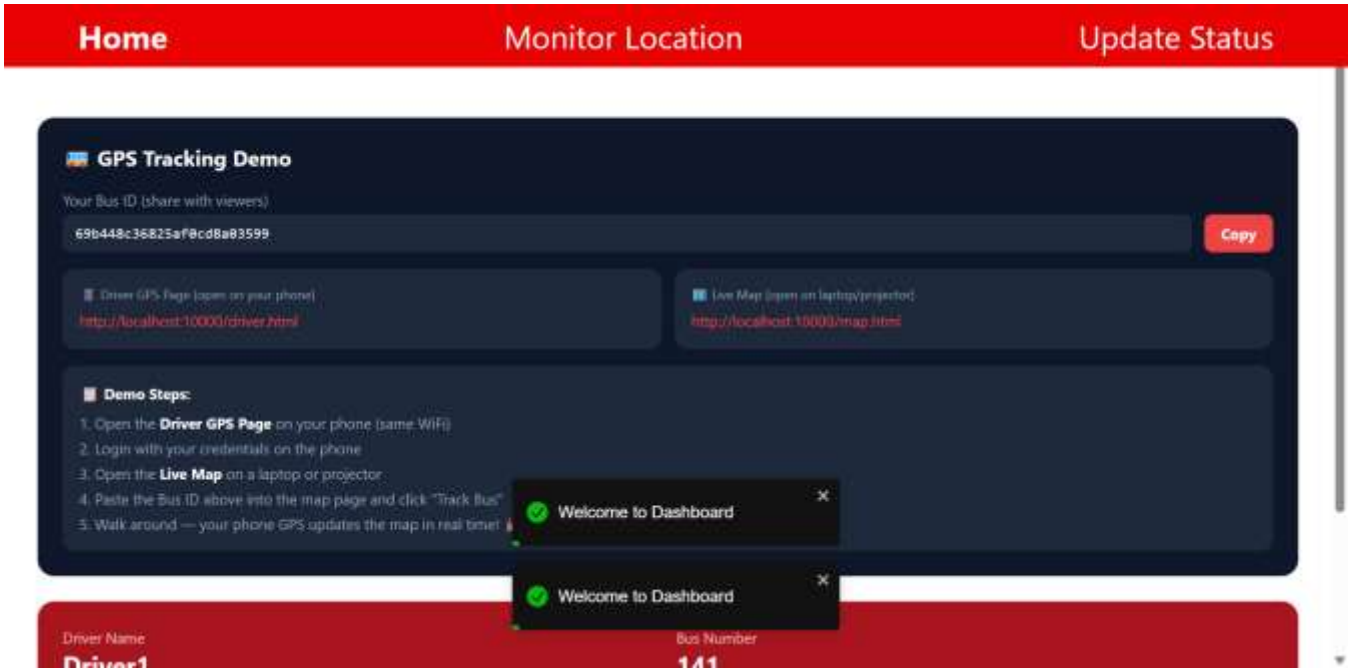
```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> net start MongoDB
The MongoDB Server (MongoDB) service is starting.
The MongoDB Server (MongoDB) service was started successfully.

PS C:\WINDOWS\system32>
```

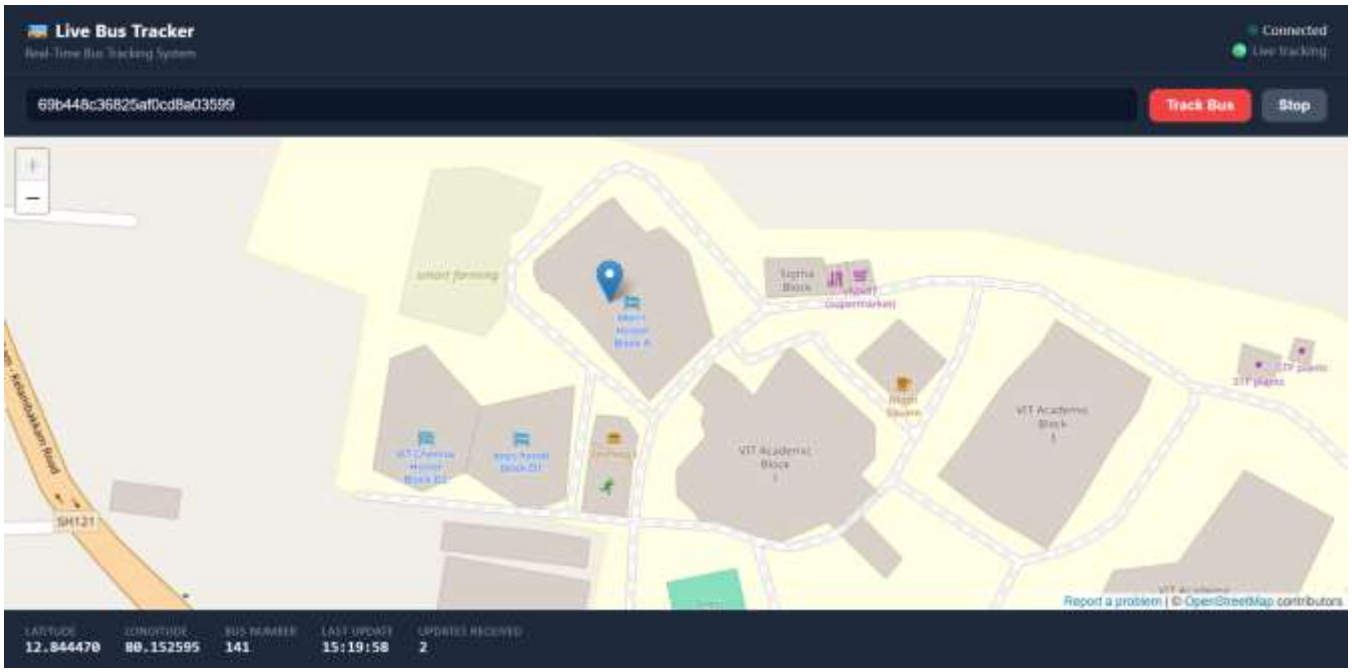
```
C:\WINDOWS\system32\cmd.
PS D:\se_project\bus-tracking\backend> npm install

up to date, audited 195 packages in 1s

20 packages are looking for funding
  run `npm fund` for details

21 vulnerabilities (6 low, 1 moderate, 13 high, 1 critical)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS D:\se_project\bus-tracking\backend> npm start

> backend@1.0.0 start
> nodemon -r dotenv/config --experimental-json-modules server.js

[nodemon] 3.0.1
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node -r dotenv/config --experimental-json-modules server.js`
mongodb://localhost:27017/tracker
Database connected successfully
hello
Server running successfully
```

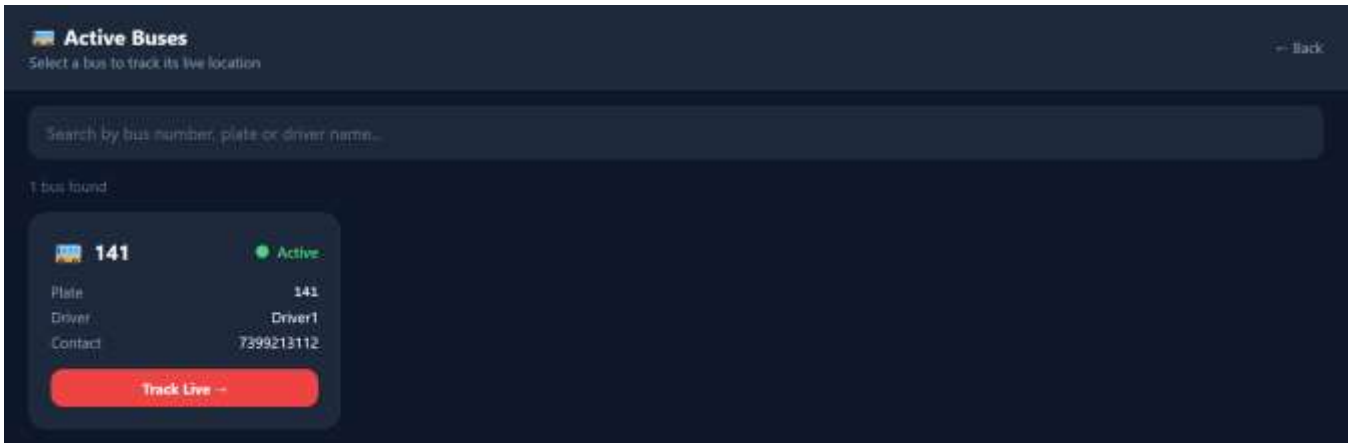
```
C:\WINDOWS\system32\cmd.

  VITE v4.4.9  ready in 2859 ms

  ➜  Local:   http://localhost:5173/bus-real
  ➜  Network: use --host to expose
  ➜  press h to show help
Browserslist: caniuse-lite is outdated. Please run:
  npx update-browserslist-db@latest
  Why you should do it regularly: https://github.com/browserslist/update-db#readme
```

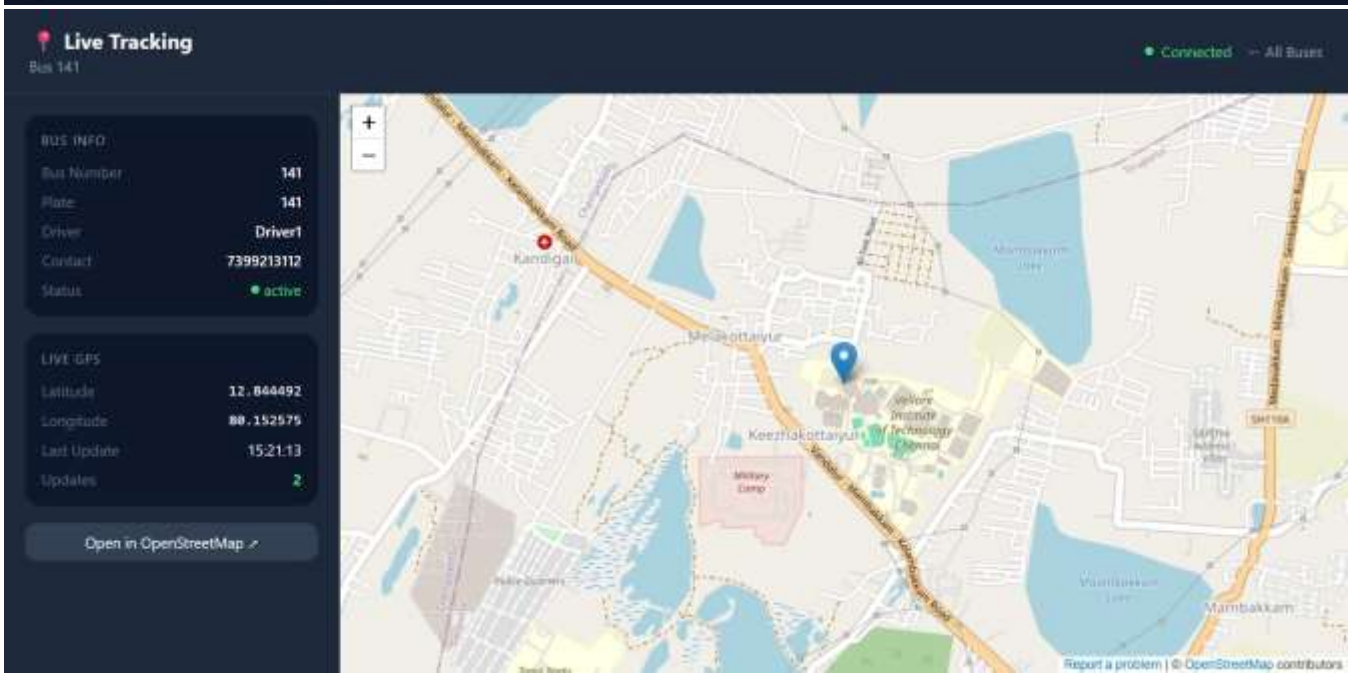
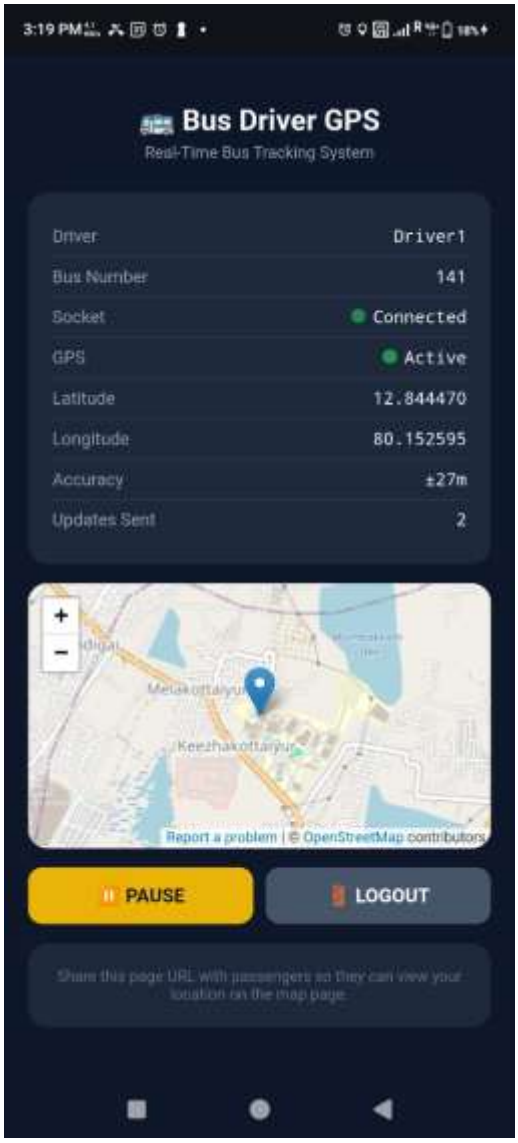
Running the MongDB service and backend and frontend severs.

BusTracker
Real-Time Bus Tracking System
Driver
Passenger


Home
Monitor Location
Update Status
GPS Tracking Demo
Your Bus ID (share with viewers)
69b448c36825af0cd8a83599
Copy
Demo Steps:
1. Open the Driver GPS Page on your phone (same WiFi)
2. Login with your credentials on the phone
3. Open the Live Map on a laptop or projector
4. Paste the Bus ID above into the map page and click "Track Bus"
5. Walk around — your phone GPS updates the map in real time!
Welcome to Dashboard
Welcome to Dashboard
Driver Name
Bus Number


Live Bus Tracker
Real-Time Bus Tracking System
Connected
Live tracking
69b448c36825af0cd8a03599
Track Bus
Stop
Report a problem | © OpenStreetMap contributors
LATITUDE
12.844470
LONGITUDE
80.152595
BUS NUMBER
141
LAST UPDATE
15:19:58
UPDATES RECEIVED
2


Active Buses
Select a bus to track its live location
← Back
Search by bus number, plate or driver name...
1 bus found
141
Active
Plate
141
Driver
Driver1
Contact
7399213112
Track Live →


Live Tracking
Bus 141
Connected
← All Buses
BUS INFO
Bus Number
141
Plate
141
Driver
Driver1
Contact
7399213112
Status
active
LIVE GPS
Latitude
12.844492
Longitude
80.152575
Last Update
15:21:13
Updates
2
Open in OpenStreetMap ↗
Report a problem | © OpenStreetMap contributors

These are screenshots from the frontend for both users and drivers. And the following are how it works on mobile devices for drivers and conductors of the buses.